Business Glossary & Stewardship Program — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Business Glossary & Stewardship Program (Microsoft Purview) 5
- The problem . . . 5
- The solution . . . 5
- How it works . . . 5
- What you get . . . 5
- The boundary . . . 5
- Bottom line . . . 6

1. Solution Overview — Business Glossary & Stewardship Program (Microsoft Purview) 6
- What this solution is . . . 6
- Why this matters . . . 6
- The four building blocks . . . 6
- Roles and least privilege . . . 7
- Pilot-first, simulation where available — safe by design . . . 7
- Licensing and the evolving portal . . . 7
- What this is not . . . 7
- The deployment model . . . 8

2. Requirements Guide — Business Glossary & Stewardship Program (Microsoft Purview) 8
- R1. Microsoft Purview Data Governance (Unified Catalog) available in your tenant . . . 8
- R2. Governance-role assignments (least privilege) for glossary curation . . . 8
- R3. A named stewardship model — owners, stewards, and an approval workflow . . . 9
- R4. A pilot governance domain and a small agreed term set (Critical Data Elements) . . . 9
- R5. Data-plane access for the automation (Purview REST via Azure sign-in — no secrets) . . . 10
- R6. Change control and a stale-term policy for the upkeep cadence . . . 10

Global Requirements — Microsoft Purview Solutions 10
- 1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11
- 2. Purview roles (least-privilege) and portal access . . . 11
- 3. Security & Compliance PowerShell + Graph toolchain . . . 11
- 4. Data sources & scope in scope . . . 12
- 5. Compliance, records, privacy & legal stakeholders . . . 12
- 6. A pilot scope and change control . . . 12

3. Implementation Guide — Business Glossary & Stewardship Program (Microsoft Purview) 13
- Step 1 — Assess current state (read-only) . . . 13
- Step 2 — Design the domain, role, and workflow model . . . 13
- Step 3 — Assign least-privilege governance roles . . . 13
- Step 4 — Record the pilot scope (read-only) . . . 14

Step 5 — Configure the stewardship workflow . . . 14
Step 6 — Seed the glossary in Draft (pilot, simulate-first) . . . 14
Step 7 — Review, approve, and publish (owner-gated) . . . 14
Step 8 — Validate against the pilot gate . . . 15
Step 9 — Establish the upkeep cadence . . . 15
Step 10 — Expand across domains (enforce) . . . 15
5. Testing & Validation Guide — Business Glossary & Stewardship Program (Microsoft Purview) 16
Gate 1 — Readiness and roles . . . 16
Gate 2 — Scope recorded . . . 16
Gate 3 — Glossary seeded in Draft (no premature publishing) . . . 16
Gate 4 — Workflow blocks unapproved publishing . . . 16
Gate 5 — Published terms are owned, defined, and linked . . . 17
Gate 6 — Upkeep cadence is evidenced . . . 17
Overall pilot pass/fail . . . 17
Rollback summary . . . 17
6. Troubleshooting Guide — Business Glossary & Stewardship Program (Microsoft Purview) 17
Sign-in, endpoint, and token . . . 18
Domains, glossaries, and API surface . . . 18
Scope and seeding . . . 19
Workflow and publishing . . . 19
Upkeep and reporting . . . 19
When a term's *meaning* is disputed . . . 20
7. Best Practices — Business Glossary & Stewardship Program (Microsoft Purview) 20
Start with Critical Data Elements, not the whole dictionary . . . 20
Every term has one owner and one steward — no exceptions . . . 20
Draft first, publish on approval — never publish unreviewed . . . 20
Definitions are business language, links are the anchor . . . 20
Separate the *definition* from the *determination* . . . 21
Least-privilege governance roles, scoped to the domain . . . 21
Re-certification is the heartbeat — schedule it . . . 21
Status hygiene: Draft → Review → Approved → Published → Expired . . . 21
Automate the read paths, gate the writes . . . 21
Verify the evolving portal and API before you rely on a path . . . 21
Make coverage visible to leadership . . . 21
8. Deliverables Summary — Business Glossary & Stewardship Program (Microsoft Purview) 22
What you receive . . . 22

Documentation set 22
PowerShell tools (5) 23
Outcomes 23
Boundary 23
Appendix A — PowerShell Scripts 25

Executive Summary — Business Glossary & Stewardship Program (Microsoft Purview)

The problem

Core business terms — "customer", "active account", "revenue", "churn" — mean different things to different teams. Reports disagree, dashboards contradict, and no one can say which number is right because no one *owns* the term. This is both a **compliance gap** (data whose meaning is contested cannot be governed) and a **data-risk exposure** (decisions are made on numbers that mean different things to the people reading them).

The solution

Stand up a **business glossary** and a **data-stewardship program** in Microsoft Purview's **Unified Catalog**: agreed **terms and critical data elements**, each with a **named owner and steward**, an approved **definition**, and a **workflow** that keeps them current. Meaning becomes shared; every term becomes owned; and an upkeep cadence keeps the glossary trustworthy instead of stale.

How it works

Delivered as a self-guided package — implementation steps plus five PowerShell tools — following a disciplined, low-risk model:

> ***Assess → Design → Configure → Pilot (in Draft) → Validate → Expand (Publish).***

New terms start in **Draft** (not business-visible), are reviewed and approved by the data owner, and only then **Published**. The tools are **read-only or `-WhatIf` by default**, with any change gated behind an explicit `-Commit` and **no secrets in code**. The pilot proves the full cycle on a small set of the highest-value terms before anything scales.

What you get

- A **governance domain** with least-privilege roles and a named stewardship RACI.
- A **glossary** of agreed, owned, defined, approved terms / critical data elements.
- A **term-approval workflow** that blocks unreviewed publishing.
- A **repeatable upkeep cadence** with reporting evidence the program is maintained.
- A validated pilot and a clear path to expand across domains.

The boundary

This solution provides the **technical method and operating model** — it does not make the business, regulatory, or records determinations. The **authoritative meaning** of a term, whether an element is a regulated critical data element, and any records/legal consequence of a definition remain with the organization's data owners, compliance, records, privacy, and legal functions. Microsoft Purview capabilities are **licensing-/plan-dependent** and the portal is **evolving** — availability, portal paths, and API

versions are verified for your tenant, never assumed.

Bottom line

Give data shared meaning and owners — delivered, validated, and defensible in Microsoft Purview, at a fraction of the cost and time of a bespoke consulting engagement.

1. Solution Overview — Business Glossary & Stewardship Program (Microsoft Purview)

What this solution is

Microsoft Purview has two related but distinct sides. The **compliance** side (sensitivity labels, DLP, retention, insider risk) governs Microsoft 365 content. This solution works on the **data governance** side — specifically the **business glossary** and the **stewardship program** that sit inside the Purview **Unified Catalog**. It gives your data *shared meaning* (agreed business terms and critical data elements) and *named owners* (data owners and data stewards), wrapped in a **workflow** so terms are proposed, reviewed, approved, and kept current rather than defined once and forgotten. It is the human-and-metadata layer that makes a data estate usable by the business, not just visible to IT.

> ***Boundary.*** *This solution provides the technical method to *stand up a glossary, define stewardship roles and workflows, assign owners, and keep terms current* in Microsoft Purview. It does **not** make the business, regulatory, or records determinations — *what a term officially means*, *which data is "confidential"*, *what the retention or records schedule is*, or *who may access what*. Those belong to the organization's data owners, stewards, compliance, privacy, and legal functions. The solution gives them the governed structure to record and enforce those decisions; it never invents the decision.*

Why this matters

"Customer", "active account", "revenue", "churn" — ask five teams and you get five definitions. Finance counts a customer one way, Sales another, Support a third. Reports disagree, dashboards contradict each other, and nobody can say which number is right because nobody owns the term. Left unaddressed this is both a **compliance gap** (you cannot govern data whose meaning is contested) and a **data-risk exposure** (decisions get made on numbers that mean different things). A business glossary fixes the *meaning* problem; a stewardship program fixes the *ownership and upkeep* problem. Together they turn a pile of ambiguous fields into a set of agreed, owned, maintained terms the whole organization can trust.

The four building blocks

This solution stands up four things in order. Each is named once in the Implementation Guide and reused everywhere.

1. **Governance domain(s)** — the business grouping inside the Unified Catalog that a glossary and its stewards belong to (for example Finance, Sales, HR). A domain carries its own ownership and roles, so glossary curation is scoped and least-privilege.
1. **The business glossary** — the set of **terms** and **critical data elements (CDEs)**: each term

has a name, an agreed definition, a **status** (Draft → Published → Expired), an **owner** and a **steward**, and — where useful — links to the data assets it describes. This is the shared dictionary of the business.

1. **The stewardship program** — the named **roles** and the **workflow** around the glossary:

governance domain owners, **data owners**, and **data stewards** who propose, review, approve, publish, and periodically re-certify terms. Roles and an approval workflow are what keep the glossary alive instead of stale.

1. **Curation & upkeep cadence** — the recurring rhythm (review stale terms, re-certify Published

terms, retire Expired ones, fill unowned gaps) that a steward runs, evidenced by the reporting script. Governance that is not maintained decays; the cadence is a first-class deliverable.

Roles and least privilege

Curation in the Unified Catalog is organized by **governance roles** assigned at the **governance domain** level: **governance domain owner** (owns the domain and its role assignments), **data owner** (accountable for a term or data product), **data steward** (curates terms, definitions, and status day to day), and **data reader** (browses the catalog). Collection-level roles in the Data Map (**collection admin**, **data curator**) may also apply where the glossary links to scanned assets. Least-privilege role assignment is an explicit step in the Implementation Guide, not an afterthought — each role is named at the stage it is needed, and account-level administration is reserved for a minimal named group.

Pilot-first, simulation where available — safe by design

Glossary and stewardship changes are lower-blast-radius than DLP or retention enforcement (creating a Draft term modifies no user data), but the same discipline applies: **pilot on a bounded domain and a small agreed term set first**, keep new terms in **Draft** status until the owner approves, and only then **Publish** and **expand** to further domains. The provided scripts are **read-only or `-WhatIf` by default**; any write is gated behind an explicit `-Commit`, and **no secret is ever stored in a script or document**.

Licensing and the evolving portal

The **business glossary** and **Unified Catalog** are part of Microsoft Purview **Data Governance**, which is billed through a **consumption / pay-as-you-go** model tied to an **Azure subscription**, and the capability set is **actively evolving** — feature names (governance domains, data products, data quality), portal paths, and the underlying data-plane API versions may have moved or may be in preview. **Verify current availability, pricing, portal paths, and API versions for your tenant** in Microsoft's documentation and in the Purview portal before you plan. This solution cites the method; it never asserts a price, a portal path, or an API version as permanent.

What this is not

This is the *shared-meaning and stewardship* solution. It is **not** the Microsoft 365 compliance configuration (sensitivity/retention labels, DLP, records management), and it is **not** the full Data Map / scanning build — those are separate Purview solutions in this category. It complements them: a glossary term can be *linked* to scanned assets and to sensitivity classifications, but standing up the estate-wide scan is out of scope here. This solution focuses squarely on giving terms meaning, owners, and a workflow.

The deployment model

> *Assess readiness and the term landscape, design the domain / role / workflow model, configure the glossary and stewardship program, **pilot on a bounded domain with a small term set (terms in Draft)**, validate against the gate, then expand across domains — publishing broadly and enforcing the upkeep cadence only after the pilot passes. The repeatable way: **Assess** → **Design** → **Configure** → **Pilot (simulate/Draft)** → **Validate** → **Expand.***

2. Requirements Guide — Business Glossary & Stewardship Program (Microsoft Purview)

Meet these before starting the Implementation Guide. The six **category** prerequisites in *Global Requirements — Microsoft Purview Solutions* (tenant/licensing, least-privilege roles, the PowerShell/Graph toolchain, in-scope data, compliance/records/privacy/legal stakeholders, and a pilot scope with change control) apply in full and are **not** repeated here. Below are the prerequisites **specific to standing up a business glossary and stewardship program**, each with *what it is*, *why it matters*, *how to verify*, and *how to implement if missing*.

> ***Determinations are deferred.** Every requirement below is about the *technical and organizational readiness* to run a glossary. The **official meaning** of a term, whether a data element is a regulated/critical data element, and any records or legal implication of a definition are set by the business data owners, compliance, records, privacy, and legal — not by this solution.*

R1. Microsoft Purview Data Governance (Unified Catalog) available in your tenant

• **Description:** The **Purview Data Governance** experience — the **Unified Catalog** with

governance domains and the **business glossary** — must be available and enabled for your tenant, tied to an **Azure subscription** for its consumption billing.

• **Why it matters:** The glossary and stewardship roles live in the Unified Catalog; without it there

is nowhere to create governance domains, terms, or governance-role assignments.

• **How to verify:** In the **Microsoft Purview portal** (purview.microsoft.com) confirm the

Unified Catalog / **Data Governance** experience loads and that you can open **Governance domains** and **Glossary**. Confirm an Azure subscription is associated for billing. Because the experience is evolving, **verify current availability and naming for your tenant**.

• **How to implement if missing:** Enable Purview Data Governance and associate an Azure subscription

per Microsoft's current onboarding documentation before starting.

R2. Governance-role assignments (least privilege) for glossary curation

• **Description:** The least-privilege **governance roles** the program needs: **governance domain

owner **(owns the pilot domain and its role assignments),** data owner **(accountable for terms),** data steward **(day-to-day curation), and** data reader **(browse). Where the glossary links to scanned assets,** collection admin **/** data curator** in the Data Map may also apply.

• **Why it matters:** Curation is gated by governance role at the domain scope; the right person needs the matching least-privilege role and nothing more.

• **How to verify:** In the Unified Catalog, open the target **governance domain** → **Roles** and confirm the named people hold domain owner / data owner / steward as intended.

• **How to implement if missing:** Assign the least-privilege governance roles at the **narrowest** scope (the pilot domain) under change control; reserve any account/tenant admin for a minimal group.

R3. A named stewardship model — owners, stewards, and an approval workflow

• **Description:** A documented **stewardship model** for the pilot: who **owns** each term (data owner), who **stewards** it (data steward), and the **workflow** by which a term moves Draft → reviewed → approved → **Published**, plus a **re-certification cadence** for keeping terms current.

• **Why it matters:** A glossary without named owners and an upkeep workflow goes stale within weeks; the program *is* the roles and the workflow, not just the term list.

• **How to verify:** Confirm a written RACI (owner vs steward vs approver) for the pilot domain and an agreed review cadence (for example, re-certify Published terms quarterly).

• **How to implement if missing:** Agree the owner/steward/approver mapping and the cadence with the business data owners before configuring workflows. *The determination of who is the authoritative owner of a business concept is the business's, not this solution's.*

R4. A pilot governance domain and a small agreed term set (Critical Data Elements)

• **Description:** One **pilot governance domain** (for example `DGGLOSS-Domain-Pilot`, such as Finance) and a **small agreed list of 5–10 terms / critical data elements (CDEs)** to seed the glossary — each with a proposed name and definition to review.

• **Why it matters:** The pilot proves the whole cycle (propose → review → approve → publish → report) on a bounded, high-value set before scaling; CDEs are where shared meaning matters most.

• **How to verify:** Confirm the pilot domain is chosen and that the 5–10 candidate terms with draft definitions are captured (a spreadsheet or list is enough to start).

• **How to implement if missing:** Work with the pilot domain's data owner to select the 5–10 most contested or highest-value terms and draft their candidate definitions before configuring.

R5. Data-plane access for the automation (Purview REST via Azure sign-in — no secrets)

• **Description:** A workstation with the **Az PowerShell** modules (`Az.Accounts`) and an admin account that can sign in with `Connect-AzAccount` and hold at least **data reader** (and, for the configuration script, **data steward / governance domain owner**) on the pilot domain. The scripts acquire a **Purview data-plane token** at runtime — **no secret is stored in code**. (The category toolchain requirement — Security & Compliance PowerShell / Graph — still applies for neighboring Purview work; the glossary automation specifically uses the Purview data-plane REST.)

• **Why it matters:** The provided Discovery, Scope, Configuration, Validation, and Reporting scripts call the Purview data-plane REST API using a token acquired from your interactive sign-in.

• **How to verify:** `Connect-AzAccount` succeeds, and `Get-AzAccessToken -ResourceUrl https://purview.azure.net` returns a token; the Purview portal shows your role on the pilot domain.

• **How to implement if missing:** Install `Az.Accounts` (`Install-Module Az.Accounts -Scope CurrentUser`), confirm modern-auth/MFA sign-in, and confirm your governance role before running any write step. **Verify the current Purview data-plane API versions for your tenant** — they evolve.

R6. Change control and a stale-term policy for the upkeep cadence

• **Description:** A lightweight **change-control** path for publishing/retiring terms and an agreed **stale-term policy** (for example, a Published term not re-certified within *N* months is flagged for review and may be set **Expired**).

• **Why it matters:** Publishing a term is an outward-facing act (the business will rely on it); an agreed review/retire policy keeps the glossary trustworthy over time and gives the reporting script a gate to measure against.

• **How to verify:** Confirm an approval path for publishing/retiring terms and a written re-certify interval.

• **How to implement if missing:** Agree the approval path and the stale-term interval with the data owners and compliance before Publishing beyond the pilot.

*Everything above is technical and organizational readiness. The **authoritative meaning** of any term, whether an element is a regulated critical data element, and any records/legal consequence of a definition remain with the business data owners, compliance, records, privacy, and legal — this solution provides the structure to record and operate their decisions, never the decision itself.*

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a

solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.

- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.
- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with stakeholders.
- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal** stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).
- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements, not technical defaults; several capabilities have HR/legal sensitivity.
- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.
- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.
- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.
- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant

governance functions.*

3. Implementation Guide — Business Glossary & Stewardship Program (Microsoft Purview)

Follow these steps in order. This guide stands up a **governance domain** and its **roles**, defines the **stewardship workflow**, seeds the **business glossary** with a small agreed term set **in Draft**, validates the cycle on a pilot, and only then **Publishes** and **expands** to further domains. Artifact names are fixed — create them with the exact names shown and reuse them everywhere. **Every new term starts in Draft; nothing is Published until the owner approves it against the pilot gate.**

> *Model: build the domain and roles, prove the propose → review → approve → publish → report cycle on a bounded term set, then expand. The repeatable way: **Assess** → **Design** → **Configure** → **Pilot (Draft)** → **Validate** → **Expand (Publish).** Verify current portal paths, capability availability, and data-plane API versions for your tenant as you go — the Purview Data Governance experience is evolving.*

Step 1 — Assess current state (read-only)

1. Run `DGGLOSS-Discovery.ps1` to capture readiness signals: the Purview account data-plane endpoint

is reachable, which **governance domains** exist, how many **glossary terms** are already present and their status mix, and whether governance roles (domain owner / steward) are assigned. No changes are made.

1. Record gaps against the Requirements checklist. Do **not** proceed past a missing Unified Catalog

experience, missing least-privilege governance roles, or a missing pilot domain — remediate first.

Step 2 — Design the domain, role, and workflow model

1. Decide the **pilot governance domain** named `DGGLOSS-Domain-Pilot` (for example, Finance). Plan

the broader domain map (by business unit) on paper for the expansion phase.

1. Decide the **stewardship RACI** for the pilot: the **data owner** (accountable for term meaning),

the **data steward** (day-to-day curator), and the **approver** for publishing. Record it.

1. Design the **term workflow**: `Draft` (proposed) → `Review` (steward + owner review) → `Approved`

→ `Published` (business-visible) → `Expired` (retired/superseded), with a **re-certification interval** (for example, re-certify Published terms every quarter).

Step 3 — Assign least-privilege governance roles

1. In the Unified Catalog, on `DGGLOSS-Domain-Pilot`, assign: **governance domain owner** to a small

named group, **data owner** to the accountable business owner, **data steward** to the curator(s), and **data reader** to browsers. Reserve any account/tenant admin for a minimal named group.

1. Where the glossary will link to scanned assets, confirm the relevant **Data Map** roles

(collection admin / data curator) exist; if not linking assets in the pilot, skip.

1. Pass/fail: each named person can perform only their intended action in the portal (a steward can

edit a Draft term; a reader cannot). Re-run `DGGLOSS-Discovery.ps1` and confirm role coverage.

Step 4 — Record the pilot scope (read-only)

1. Run `DGGLOSS-Set-Scope.ps1` to record the pilot **domain**, the 5–10 candidate **terms / critical

data elements**, the** owner/steward **mapping, and the** re-certification interval**. This is read-only and produces the scope manifest you configure against.

1. Confirm with the pilot domain's **data owner** that the candidate term list and draft definitions

are the right starting set. *The authoritative meaning of each term is the owner's to set.*

Step 5 — Configure the stewardship workflow

1. In the Unified Catalog, configure (or confirm) the **term-approval workflow** for

`DGGLOSS-Domain-Pilot` so that creating/updating a term routes to the named **approver** before it can be **Published**. Where a built-in approval workflow is available in your tenant, use it; where it is not, record the manual approval step in the RACI. **Verify workflow availability for your tenant** — this feature is evolving.

1. Set the **status model** convention (Draft/Review/Approved/Published/Expired) that the team will

use consistently.

1. Pass/fail: a test Draft term cannot reach **Published** without passing the approver. Rollback:

there is nothing to undo — a Draft term is not business-visible.

Step 6 — Seed the glossary in Draft (pilot, simulate-first)

1. Create `DGGLOSS-Domain-Pilot` if it does not exist and seed the agreed terms **in Draft** using

`DGGLOSS-Configure-Glossary.ps1`, which **defaults to** `-WhatIf` and prints exactly the domain and each term it *would* create, changing nothing. Review that plan.

1. Re-run with `-Commit` to create the domain and the terms **in** `Draft` **status**, each carrying its

proposed **definition**, an **owner**, and a **steward**. No term is Published in this step.

1. Pass/fail: `DGGLOSS-Discovery.ps1` / the portal show the pilot domain with the seeded terms present

in **Draft**, each with a definition and an owner. Rollback: delete or leave Draft terms — they are not business-visible; re-run after correcting definitions.

Step 7 — Review, approve, and publish (owner-gated)

1. The **data steward** reviews each Draft term with the **data owner**; the owner sets the

authoritative definition and approves. Move approved terms to **Published** through the workflow (portal or `DGGLOSS-Configure-Glossary.ps1 -SetStatus Published -Commit`, owner-approved).

1. Link terms to their **critical data elements / assets** where useful so the meaning is anchored to real data.

1. Pass/fail: each Published term has an approved definition, a named owner and steward, and (where applicable) linked assets. Only owner-approved terms are Published.

Step 8 — Validate against the pilot gate

1. Run `DGGLOSS-Validate.ps1` and complete the Testing Guide. The pilot passes only when: the pilot domain exists with least-privilege roles assigned; every seeded term has a **definition**, a named **owner**, and a **steward**; the **workflow** blocks unapproved publishing; and the agreed pilot terms are **Published** (the rest legitimately still Draft/Review).

1. Remediate any gap and re-run the relevant validation before widening scope. Do **not** expand to further domains until the gate passes.

Step 9 — Establish the upkeep cadence

1. Run `DGGLOSS-Report.ps1` to produce the stewardship report: terms by **status**, **unowned** terms, **stale** Published terms past the re-certification interval, and coverage per domain.

1. Schedule the steward's recurring cadence (for example monthly): re-certify Published terms, retire superseded terms to **Expired**, and fill unowned/undefined gaps. The report is the evidence the program is being maintained.

Step 10 — Expand across domains (enforce)

1. After the pilot passes, add governance domains one at a time (`DGGLOSS-Domain-<name>`), re-running `DGGLOSS-Set-Scope.ps1`, `DGGLOSS-Configure-Glossary.ps1` (Draft-first, `-WhatIf` then `-Commit`), `DGGLOSS-Validate.ps1`, and `DGGLOSS-Report.ps1` for each new domain.

1. Keep the discipline: new terms start **Draft**, owner-approved before **Published**, every term owned and stewarded, and the upkeep cadence enforced with `DGGLOSS-Report.ps1` per domain.

1. Retire and re-certify on the agreed interval so the glossary stays trustworthy as the business evolves.

Order of operations

```
Assess -> Design (domain/roles/workflow) -> Assign roles -> Record scope ->
Configure workflow -> Seed glossary (Draft, -WhatIf then -Commit) ->
Review/approve/Publish (owner-gated) -> Validate gate -> Upkeep cadence ->
```

```
Expand (further domains)
```

5. Testing & Validation Guide — Business Glossary & Stewardship Program (Microsoft Purview)

Validation is a set of **pass/fail gates**. The pilot is not "done" until every gate below passes on `DGGLOSS-Domain-Pilot`. Run the read-only scripts to gather evidence; capture the output as the validation record. Where a check depends on a portal feature that is evolving, **verify the current path and availability for your tenant**.

> *Principle: a term is trustworthy only when it has an **agreed definition**, a **named owner and steward**, an **approved status**, and it is **kept current**. The gates below test exactly those properties — not merely that a term row exists.*

Gate 1 — Readiness and roles

• **Test:** Run `DGGLOSS-Discovery.ps1`. Confirm the Unified Catalog endpoint is reachable, the pilot domain exists (or is ready to create), and governance roles (domain owner / data owner / steward) are assigned at the domain scope.

• **Pass:** Endpoint PASS; `DGGLOSS-Domain-Pilot` present; each governance role shows assigned.

• **Fail:** Endpoint unreachable, domain missing, or a role shows unassigned → remediate per Requirements R1/R2 before continuing.

Gate 2 — Scope recorded

• **Test:** Run `DGGLOSS-Set-Scope.ps1` and confirm the manifest lists the pilot domain, the 5–10 candidate terms, the owner/steward mapping, and the re-certification interval.

• **Pass:** A complete scope manifest is produced and matches what the data owner agreed.

• **Fail:** Missing terms, owners, or interval → complete the scope with the data owner and re-run.

Gate 3 — Glossary seeded in Draft (no premature publishing)

• **Test:** After `DGGLOSS-Configure-Glossary.ps1 -WhatIf` then `-Commit`, run `DGGLOSS-Report.ps1`.

• **Pass:** All seeded terms exist in **Draft** status, each with a definition, an owner, and a steward; **none** is Published yet.

• **Fail:** A term is missing a definition/owner/steward, or a term was Published before owner approval → correct and re-run. (A Draft term is not business-visible; safe to fix.)

Gate 4 — Workflow blocks unapproved publishing

• **Test:** Attempt to move a test Draft term to **Published** as a **data steward** (not the approver). Then have the **approver** approve and publish it.

• **Pass:** The steward alone cannot Publish; only after approver approval does the term reach **Published**.

• **Fail:** A term reaches Published without approver sign-off → fix the workflow / manual gate per Implementation Step 5 before Publishing further. **Verify approval-workflow availability for your tenant.**

Gate 5 — Published terms are owned, defined, and linked

• **Test:** Run `DGGLOSS-Validate.ps1` against the pilot domain.

• **Pass:** Every **Published** pilot term has an approved definition, a named owner and steward, and (where applicable) at least one linked critical data element / asset.

• **Fail:** Any Published term missing a definition, owner, steward, or intended link → return it to Draft/Review, correct, re-approve.

Gate 6 — Upkeep cadence is evidenced

• **Test:** Run `DGGLOSS-Report.ps1` and review the stewardship report.

• **Pass:** The report shows: term counts by status; **zero** unowned Published terms; and no Published term past the re-certification interval (or each such term flagged for review). The steward cadence (e.g. monthly) is scheduled.

• **Fail:** Unowned Published terms, or stale terms with no review action → assign owners / re-certify or retire (**Expired**) before declaring the program operational.

Overall pilot pass/fail

The pilot **passes** only when **Gates 1–6 all pass** on `DGGLOSS-Domain-Pilot`. Record the script outputs as the validation evidence pack. Do **not** expand to further governance domains until the pilot gate is green. On expansion, re-run Gates 1–6 per new domain.

Rollback summary

Glossary changes are low-blast-radius: a **Draft** term is not business-visible, so the safe rollback for any misconfiguration is to keep the term in **Draft** (or delete it) and correct the definition, owner, or link before re-approving. Retiring a bad Published term is a status change to **Expired**, not a data deletion. No user data is modified by any step in this solution.

6. Troubleshooting Guide — Business Glossary & Stewardship Program (Microsoft Purview)

Symptom → cause → resolution for the common issues when standing up and running the glossary and stewardship program. Because Purview Data Governance is **evolving**, several fixes end in "verify the current path / API version for your tenant" — that is expected, not a defect.

Sign-in, endpoint, and token

`DGGLOSS-Discovery.ps1` **reports the endpoint as GAP / not reachable.**

- *Cause:* wrong account name, no role on the account, or a moved account API version.
- *Resolution:* confirm the short `-PurviewAccount` name resolves to

`https://<name>.purview.azure.com`; confirm you hold at least **Data reader** on the target domain; and **verify the current Atlas glossary / account API version for your tenant** — the example versions in the scripts are illustrative.

"Not signed in. Run Connect-AzAccount first." / token acquisition fails.

- *Cause:* no Azure context, or the `Az.Accounts` module is missing.
- *Resolution:* `Install-Module Az.Accounts -Scope CurrentUser`, then `Connect-AzAccount` with a

modern-auth/MFA-capable admin. Confirm `Get-AzAccessToken -ResourceUrl https://purview.azure.net` returns a token. The scripts never store a secret — they acquire the token from your sign-in.

A write returns 401/403 even though sign-in works.

- *Cause:* the account can read but lacks the **governance role** to write (data steward / governance

domain owner) on the domain.

- *Resolution:* assign the least-privilege write role at the **domain scope** (Requirements R2), then

re-run `-Commit`. Do not escalate to tenant admin to "make it work".

Domains, glossaries, and API surface

`DGGLOSS-Discovery.ps1` **shows "Governance domns: GAP/verify".**

- *Cause:* the Unified Catalog governance-domain API path/version has moved, or Data Governance is not

enabled for the tenant.

- *Resolution:* confirm the **Unified Catalog** loads in the portal (Requirements R1) and **verify the

current governance-domain API version** for your tenant; update the `api-version` in the script to match. Glossary operations can still proceed via the Atlas glossary API even while the newer governance-domain API differs.

`DGGLOSS-Configure-Glossary.ps1 -SeedTerms` **says "No glossary found".**

- *Cause:* there is no glossary to anchor terms to yet.
- *Resolution:* create the glossary/governance domain in the Unified Catalog first (portal), then

re-run; or pass `-GlossaryGuid` explicitly. Verify the current create path for your tenant.

Scope and seeding

Configure refuses to seed: "Scope has N incomplete term(s)".

- *Cause:* one or more candidate rows is missing name, definition, owner, or steward.
- *Resolution:* this is the intended gate — complete the rows with the data owner and re-run `DGGLOSS-Set-Scope.ps1`, then seed. A term without an owner and definition must not be created.

Terms were created but are not visible to the business.

- *Cause:* correct behavior — new terms are created in **Draft**, which is not business-visible.
- *Resolution:* have the owner approve and move them to **Published** (Implementation Step 7). Do not bypass the approval gate to make them visible.

Workflow and publishing

A steward was able to Publish a term without approval.

- *Cause:* the term-approval workflow is not configured (or the tenant lacks the built-in workflow) and the manual gate was skipped.
- *Resolution:* configure the approval workflow for the domain, or enforce the documented manual approval step in the RACI (Implementation Step 5). **Verify approval-workflow availability for your tenant** — where it is unavailable, the manual gate is mandatory. Return the wrongly-Published term to Draft, correct, re-approve.

`DGGLOSS-Validate.ps1` **reports GATE: FAIL — Published-incomplete > 0.**

- *Cause:* a Published term is missing a definition, owner, or steward — it should never have reached Published.
- *Resolution:* set the term back to **Draft** (`-SetStatus Draft -Commit`), fix the missing property, re-approve, and re-Publish. Re-run `DGGLOSS-Validate.ps1` until the gate is PASS.

Upkeep and reporting

`DGGLOSS-Report.ps1` **shows unowned Published terms (> 0).**

- *Cause:* a term was Published without an owner recorded, or an owner left and was not reassigned.
- *Resolution:* assign a data owner immediately — treat this as a defect, not a nuance. The target is **zero** unowned Published terms.

Report shows many "stale Published terms".

- *Cause:* Published definitions have not been re-certified within the interval.
- *Resolution:* run the steward cadence — re-certify current terms, retire superseded ones to **Expired** (`-SetStatus Expired -Commit`). Consider shortening the cadence if staleness recurs.

Stale detection looks wrong / timestamps missing.

- *Cause:* the term's `updateTime` field is absent or in a different shape than expected in your tenant's API response.
- *Resolution:* the report degrades safely (a term with no timestamp is simply not flagged stale); **verify the current term schema / API version for your tenant** and adjust the timestamp field if Microsoft has changed it.

When a term's *meaning* is disputed

- *Cause:* two teams disagree on the authoritative definition.
- *Resolution:* this is a **business determination, not a tooling problem**. Keep the term in **Draft/Review**, escalate to the data owner (and compliance/records where a regulated element is involved) to set the authoritative meaning, then approve and Publish. The solution records the decision; it does not adjudicate it.

7. Best Practices — Business Glossary & Stewardship Program (Microsoft Purview)

Field-tested practices for a glossary that stays trustworthy. These are operational guidance, not regulatory or records determinations — those remain with the business data owners, compliance, records, privacy, and legal.

Start with Critical Data Elements, not the whole dictionary

A glossary that tries to define everything on day one defines nothing well. Seed with **5–10 Critical Data Elements** — the terms that are most contested, most reported-on, or most regulated (Customer, Active Account, Revenue, Churn). Prove the cycle on those, then expand. Breadth without upkeep is how glossaries die.

Every term has one owner and one steward — no exceptions

The single biggest failure mode is the **unowned term**. Enforce that every Published term names a **data owner** (accountable for meaning) and a **data steward** (accountable for upkeep). The `DGGLOSS-Report.ps1` "unowned Published terms" count should be **zero**; treat any non-zero as a defect, not a nuance.

Draft first, publish on approval — never publish unreviewed

Keep new and changed terms in **Draft** until the owner approves. Publishing is an outward-facing act: the business will build reports and decisions on a Published definition, so the approver gate is what makes it safe. Use the workflow (or a documented manual approval) every time.

Definitions are business language, links are the anchor

Write definitions in **plain business language** the whole organization understands — not a database column description. Then **link** the term to the actual critical data elements / assets it describes so the meaning is anchored to real data and lineage, not floating in prose.

Separate the *definition* from the *determination*

The glossary records **what a term means to the business**. It does **not** decide the *regulatory classification*, the *retention/records schedule*, or *who may access* the underlying data — those are determinations owned by compliance, records, privacy, and legal. Keep that line bright: a steward curates meaning; a governance function makes the legal/records call.

Least-privilege governance roles, scoped to the domain

Assign governance domain owner / data owner / steward / reader at the **narrowest scope** (the domain), and reserve account/tenant administration for a minimal named group. A steward should be able to curate their domain's terms and nothing else. Re-verify role coverage with `DGGLOSS-Discovery.ps1`.

Re-certification is the heartbeat — schedule it

A Published definition that is never re-reviewed silently drifts from reality. Set a **re-certification interval** (quarterly is common), and run `DGGLOSS-Report.ps1` on a monthly cadence to surface **stale** Published terms and retire superseded ones to **Expired**. An unmaintained glossary is worse than none — people trust wrong definitions.

Status hygiene: Draft → Review → Approved → Published → Expired

Use the status model consistently so the report is meaningful. **Expired** is a first-class outcome: retiring a superseded term is good stewardship, not failure. Never delete history where the term was relied upon — Expire it so past references still resolve.

Automate the read paths, gate the writes

Use the read-only scripts (`Discovery`, `Set-Scope`, `Validate`, `Report`) freely and often — they change nothing. Keep every write behind `-WhatIf` by default and an explicit `-Commit`, and **never** put a secret in a script or a document; the scripts acquire a data-plane token from your interactive Azure sign-in at runtime.

Verify the evolving portal and API before you rely on a path

Purview Data Governance — governance domains, the Unified Catalog, approval workflows, data-plane API versions — is **actively evolving**. Before you script or document a portal path or API version as "the way", **verify it for your current tenant**. This solution cites the method and flags each evolving surface; treat any specific path/version as "verify first".

Make coverage visible to leadership

The stewardship program earns its keep when leadership can see progress: terms Published per domain, percentage of CDEs defined and owned, and stale-term trend. Use `DGGLOSS-Report.ps1` output to build

a simple recurring coverage view so the program is funded and sustained.

8. Deliverables Summary — Business Glossary & Stewardship Program (Microsoft Purview)

What you receive

A complete, self-guided package to stand up a **business glossary** and a **data-stewardship program** in Microsoft Purview's **Unified Catalog** — configured, piloted in Draft, validated against explicit gates, and operable with a repeatable upkeep cadence.

Documentation set

Document	What it gives you
1. Solution Overview	What the glossary + stewardship program is, the four building blocks (governance domain, glossary, stewardship program, upkeep cadence), roles, and the Assess→Design→Configure→Pilot→Validate→Expand model.
2. Requirements Guide	The solution-specific prerequisites (Unified Catalog availability, least-privilege governance roles, the stewardship RACI + workflow, a pilot domain and CDE list, data-plane access, change control) — each with verify/implement — on top of the six category Global Requirements.
3. Implementation Guide	Exact ordered steps: design the domain/roles/workflow, seed the glossary **in Draft** (`-WhatIf` then `-Commit`), owner-gated publish, validate, run the upkeep cadence, expand — with named artifacts, pass/fail gates, and rollback.
5. Testing & Validation Guide	Six pass/fail gates from readiness through owned/defined Published terms to an evidenced upkeep cadence, plus the rollback summary.
6. Troubleshooting Guide	The common failure modes (endpoint/token, roles, workflow, unowned or stale terms, evolving API/portal) and how to resolve each.
7. Best Practices	Field-tested guidance: start with CDEs, one owner + one steward per term, Draft-first publishing, definition vs determination, least privilege, re-certification cadence, verify the evolving portal.
Executive Summary	A one-page business framing for sponsors and stakeholders.

PowerShell tools (5)

All tools are **read-only or `-WhatIf` by default**; any write is gated behind an explicit `-Commit`. They acquire a **Purview data-plane token** from your interactive `Connect-AzAccount` sign-in — **no secret is stored in code**. Verify the current Purview data-plane API versions for your tenant.

Script	Role	Safety
DGGLOSS-Discovery.ps1	Assess readiness: endpoint reachable, governance domains, glossary term count + status mix, governance-role coverage.	Read-only
DGGLOSS-Set-Scope.ps1	Record the pilot scope: domain, candidate terms/CDEs, owner/steward mapping, re-certification interval → scope manifest.	Read-only
DGGLOSS-Configure-Glossary.ps1	Configure/remediate: create the governance domain and seed terms **in Draft** (owner/steward/definition); set a term's status (e.g. Published) with `-Commit`.	`-WhatIf` default; `-Commit` to write
DGGLOSS-Validate.ps1	Validate against the pilot gate: Published terms have definition, owner, steward, and intended links; workflow gate honored.	Read-only
DGGLOSS-Report.ps1	Stewardship report: terms by status, unowned Published terms, stale terms past re-certification, coverage per domain.	Read-only

Outcomes

- A **governance domain** with least-privilege roles and a named stewardship RACI.
- A **business glossary** of agreed terms / critical data elements — each **owned**, **stewarded**, **defined**, and **approved** before it is Published.
- A **term-approval workflow** that blocks unreviewed publishing.
- A **repeatable upkeep cadence** with reporting evidence that the program is maintained.
- A validated pilot and a documented path to expand across further domains.

Boundary

The package delivers the **technical configuration and operating model**. The **authoritative meaning** of each term, whether an element is a regulated critical data element, and any records/legal consequence of a definition remain determinations of the business data owners, compliance, records, privacy, and legal functions. Purview features are licensing-/plan-dependent and the portal is evolving — verify availability, paths, and API versions for your tenant.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DGGLOSS-Configure-Glossary.ps1

Purpose: Configures the business glossary for the pilot from the scope manifest: creates each candidate term IN DRAFT status with its definition, owner, and steward (-SeedTerms), or moves a single approved term to a new status such as Published (-SetStatus). SAFE BY DEFAULT: with no -Commit it runs in -WhatIf mode and prints the exact payload it WOULD send, changing nothing. Pass -Commit to write. No secrets in code — a data-plane token is acquired from your interactive Azure sign-in. HOW TO RUN Seed candidate terms in Draft (preview, changes nothing): .\DGGLOSS-Configure-Glossary.ps1 -PurviewAccount contoso-purview -ScopeFile .\DGGLOSS-scope.json -SeedTerms Seed for real: .\DGGLOSS-Configure-Glossary.ps1 -PurviewAccount contoso-purview -ScopeFile .\DGGLOSS-scope.json -SeedTerms -Commit Publish an approved term (owner-approved only): .\DGGLOSS-Configure-Glossary.ps1 -PurviewAccount contoso-purview -TermGuid <guid> -SetStatus Published -Commit Requires Az.Accounts + Connect-AzAccount with Data steward / governance domain owner as appropriate. EXPECTED RESULTS Default: a printed "[WhatIf] would create/update ..." plan and NO change. With -Commit each term is created in Draft (or the named term's status is updated) and the script prints its name/GUID. Verify the current Atlas glossary / Unified Catalog API versions for your tenant — evolving. .NOTES Purview — Data Governance. Writes gated behind -Commit; -WhatIf default; new terms are always Draft. Determinations (what a term MEANS, who OWNS it, whether it may be Published) are the owner's — this tool records their approved decisions, it does not make them. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [switch]$SeedTerms, [string]$ScopeFile = ".\DGGLOSS-scope.json", [string]$GlossaryGuid, # target glossary; if omitted, first glossary is used status change (single term) [string]$TermGuid, [ValidateSet('Draft','Approved','Published','Expired')] [string]$SetStatus, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com", [switch]$Commit)

How to run: Seed candidate terms in Draft (preview, changes nothing): .\DGGLOSS-Configure-Glossary.ps1 -PurviewAccount contoso-purview -ScopeFile .\DGGLOSS-scope.json -SeedTerms Seed for real: .\DGGLOSS-Configure-Glossary.ps1 -PurviewAccount contoso-purview -ScopeFile .\DGGLOSS-scope.json -SeedTerms -Commit Publish an approved term (owner-approved only): .\DGGLOSS-Configure-Glossary.ps1 -PurviewAccount contoso-purview -TermGuid <guid> -SetStatus Published -Commit Requires Az.Accounts + Connect-AzAccount with Data steward / governance domain owner as appropriate. EXPECTED RESULTS Default: a printed "[WhatIf] would create/update ..." plan and NO change. With -Commit each term is created in Draft (or the named term's status is updated) and the script prints its name/GUID. Verify the current Atlas glossary / Unified Catalog API versions for your tenant — evolving. .NOTES Purview — Data Governance. Writes gated behind -Commit; -WhatIf default; new terms are always Draft. Determinations (what a term MEANS, who OWNS it, whether it may be Published) are the owner's — this tool records their approved decisions, it does not make them. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [switch]$SeedTerms, [string]$ScopeFile = ".\DGGLOSS-scope.json", [string]$GlossaryGuid, # target glossary; if omitted, first glossary is used status change (single term) [string]$TermGuid, [ValidateSet('Draft','Approved','Published','Expired')] [string]$SetStatus, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com", [switch]$Commit)

DGGLOSS-Discovery.ps1

Purpose: Reports readiness before you seed a glossary or assign stewards: whether the Microsoft Purview account data-plane endpoint is reachable, which governance domains / glossaries exist, how many glossary terms are present and their status mix (Draft/Approved/Published/Expired), and whether governance roles (domain owner / data steward) appear assigned. Read-only — it makes no change to the account, domains, glossary, or roles. Supports the Assess step. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGGLOSS-Discovery.ps1 -PurviewAccount contoso-purview Requires the Az PowerShell modules (Az.Accounts). Sign in first with Connect-AzAccount using an account holding at least Data reader on the target governance domain. The script acquires a data-plane token for the Purview REST API (resource https://purview.azure.net); no secret is stored in the script. EXPECTED RESULTS A readiness summary with a PASS/GAP line per signal: endpoint reachable, governance-domain count, glossary term count and status breakdown, and governance-role coverage — to record against the Requirements checklist before seeding terms. .NOTES Purview — Data Governance (Unified Catalog). Read-only against the Purview data-plane REST API. Verify your licensing/tenant and current API versions: the Data Governance experience is evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$PurviewAccount, # short account name, e.g. contoso-purview [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGGLOSS-Discovery.ps1 -PurviewAccount contoso-purview Requires the Az PowerShell modules (Az.Accounts). Sign in first with Connect-AzAccount using an account holding at least Data reader on the target governance domain. The script acquires a data-plane token for the Purview REST API (resource https://purview.azure.net); no secret is stored in the script. EXPECTED RESULTS A readiness summary with a PASS/GAP line per signal: endpoint reachable, governance-domain count, glossary term count and status breakdown, and governance-role coverage — to record against the Requirements checklist before seeding terms. .NOTES Purview — Data Governance (Unified Catalog). Read-only against the Purview data-plane REST API. Verify your licensing/tenant and current API versions: the Data Governance experience is evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$PurviewAccount, # short account name, e.g. contoso-purview [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

DGGLOSS-Report.ps1

Purpose: Produces the recurring stewardship report that evidences the upkeep cadence: term counts by status (Draft/Approved/Published/Expired), UNOWNED Published terms (a defect), and STALE Published terms whose last update is older than the re-certification interval. Read-only — it makes NO change; run it on a monthly cadence and keep the output as program evidence. Optionally exports a CSV. HOW TO RUN .\DGGLOSS-Report.ps1 -PurviewAccount contoso-purview -RecertifyMonths 3 Export a CSV of the term inventory: .\DGGLOSS-Report.ps1 -PurviewAccount contoso-purview -RecertifyMonths 3 -CsvOut .\DGGLOSS-report.csv Requires Az.Accounts + Connect-AzAccount with at least Data reader on the domain. A data-plane token is acquired from your sign-in; no secret is stored in the script. EXPECTED RESULTS A status breakdown, an "unowned Published terms" count (target zero), a "stale Published terms" list (past the re-certification interval), and per-glossary coverage — the evidence the program is being maintained. Non-zero unowned/stale counts are the steward's work list. .NOTES Purview — Data Governance. Read-only reporting. Verify current Atlas glossary API versions for your tenant. Retiring a superseded term to Expired and re-certifying Published terms are stewardship actions; regulatory/records determinations remain with the relevant governance functions. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [int]$RecertifyMonths = 3, [string]$CsvOut, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: .\DGGLOSS-Report.ps1 -PurviewAccount contoso-purview -RecertifyMonths 3 Export a CSV of the term inventory: .\DGGLOSS-Report.ps1 -PurviewAccount contoso-purview -RecertifyMonths 3 -CsvOut .\DGGLOSS-report.csv Requires Az.Accounts + Connect-AzAccount with at least Data reader on the domain. A data-plane token is acquired from your sign-in; no secret is stored in the script. EXPECTED RESULTS A status breakdown, an "unowned Published terms" count (target zero), a "stale Published terms" list (past the re-certification interval), and per-glossary coverage — the evidence the program is being maintained. Non-zero unowned/stale counts are the steward's work list. .NOTES Purview — Data Governance. Read-only reporting. Verify current Atlas glossary API versions for your tenant. Retiring a superseded term to Expired and re-certifying Published terms are stewardship actions; regulatory/records determinations remain with the relevant governance functions. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [int]$RecertifyMonths = 3, [string]$CsvOut, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

DGGLOSS-Set-Scope.ps1

Purpose: Captures and validates the pilot scope you will configure against: the governance domain, the 5-10 candidate terms / critical data elements, the owner/steward mapping, and the re-certification interval. Reads an optional CSV of candidate terms, checks each row is complete (name, definition, owner, steward), and writes a scope manifest JSON. Read-only with respect to Purview — it makes NO change to any domain, glossary, term, or role. HOW TO RUN .\DGGLOSS-Set-Scope.ps1 -Domain DGGLOSS-Domain-Pilot -TermCsv .\pilot-terms.csv ` -RecertifyMonths 3 -OutFile .\DGGLOSS-scope.json pilot-terms.csv columns: TermName,Definition,Owner,Steward,CriticalDataElement (No Purview sign-in is required — this step only assembles and checks the local scope manifest.) EXPECTED RESULTS A printed per-term completeness check (OK / MISSING ...), a summary count, and a scope-manifest JSON written to -OutFile for the Configure/Validate/Report steps to consume. Non-zero MISSING rows are flagged so you complete them with the data owner before configuring. .NOTES Purview — Data Governance. Read-only planning helper; assembles the scope manifest only. The authoritative MEANING/ownership of each term is the business data owner's decision, not this tool's. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$Domain, # e.g. DGGLOSS-Domain-Pilot [string]$TermCsv, # optional CSV of candidate terms [int]$RecertifyMonths = 3, # re-certification interval for Published terms [string]$OutFile = ".\DGGLOSS-scope.json")

How to run: .\DGGLOSS-Set-Scope.ps1 -Domain DGGLOSS-Domain-Pilot -TermCsv .\pilot-terms.csv ` -RecertifyMonths 3 -OutFile .\DGGLOSS-scope.json pilot-terms.csv columns: TermName,Definition,Owner,Steward,CriticalDataElement (No Purview sign-in is required — this step only assembles and checks the local scope manifest.) EXPECTED RESULTS A printed per-term completeness check (OK / MISSING ...), a summary count, and a scope-manifest JSON written to -OutFile for the Configure/Validate/Report steps to consume. Non-zero MISSING rows are flagged so you complete them with the data owner before configuring. .NOTES Purview — Data Governance. Read-only planning helper; assembles the scope manifest only. The authoritative MEANING/ownership of each term is the business data owner's decision, not this tool's. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$Domain, # e.g. DGGLOSS-Domain-Pilot [string]$TermCsv, # optional CSV of candidate terms [int]$RecertifyMonths = 3, # re-certification interval for Published terms [string]$OutFile = ".\DGGLOSS-scope.json")

DGGLOSS-Validate.ps1

Purpose: Checks the glossary against the pilot pass/fail gate: every term has a non-empty definition, a named owner and steward, and a valid status; and every PUBLISHED term is fully complete (the

outward-facing bar). Optionally validates only the terms named in a scope manifest. Read-only — it makes NO change; it reports PASS/FAIL per check so you can record the validation evidence. HOW TO RUN .\DGGLOSS-Validate.ps1 -PurviewAccount contoso-purview Scope to the pilot's terms only: .\DGGLOSS-Validate.ps1 -PurviewAccount contoso-purview -ScopeFile .\DGGLOSS-scope.json Requires Az.Accounts + Connect-AzAccount with at least Data reader on the domain. A data-plane token is acquired from your sign-in; no secret is stored in the script. EXPECTED RESULTS A per-term PASS/FAIL table (definition / owner / steward / status) plus a Published-completeness summary and an overall gate verdict. Any FAIL identifies exactly which property to fix before the pilot can pass. .NOTES Purview — Data Governance. Read-only validation. Verify current Atlas glossary API versions for your tenant. This tool tests COMPLETENESS of stewardship metadata, not the correctness of a definition — the authoritative meaning remains the data owner's determination. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$ScopeFile, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: .\DGGLOSS-Validate.ps1 -PurviewAccount contoso-purview Scope to the pilot's terms only: .\DGGLOSS-Validate.ps1 -PurviewAccount contoso-purview -ScopeFile .\DGGLOSS-scope.json Requires Az.Accounts + Connect-AzAccount with at least Data reader on the domain. A data-plane token is acquired from your sign-in; no secret is stored in the script. EXPECTED RESULTS A per-term PASS/FAIL table (definition / owner / steward / status) plus a Published-completeness summary and an overall gate verdict. Any FAIL identifies exactly which property to fix before the pilot can pass. .NOTES Purview — Data Governance. Read-only validation. Verify current Atlas glossary API versions for your tenant. This tool tests COMPLETENESS of stewardship metadata, not the correctness of a definition — the authoritative meaning remains the data owner's determination. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$ScopeFile, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")